

Networking Assignment

1. A service is running on a server and is reachable locally using curl localhost:8080 but not from another machine using curl :8080. List at least 4 possible causes, ordered from most likely to least likely.

Ans : The following are the causes :

1. Firewall Rules Issues : The server firewall might be blocking the incoming connections on the port 8080. The service might be accessible only on localhost .To correct this we can check the firewall security rules and correct / modify them if needed.
2. Localhost Binding Issue : It may be the case that the service might be bound to the loopback address i.e it accepts only request from the localhost binding it to only it i.e the localhost.
3. Network Routing Issues : There might be the issue with the way the network is routing the requests preventing the other machines to reach the server.
4. The DNS server may not have IP of the remote host present inside the records maintained by it so it will not be able to connect.

2. A TCP-based application experiences intermittent timeouts even though the server is healthy.

a. Explain three network-level reasons this could happen without any packet drops visible at the application layer.

Ans The following network-level reasons could be the causes:

1. If the server is running fine but the devices at the network- level like router , or switches may be faulty or congested which may cause the packet drops to occur . The TCP will try to re-transmit the packet before the timeout occurs.
2. Asymmetric routing can also be one of the reason for it. The request takes one path way and the response take another path way this may lead to packet drops at the background because the security exist only on one pathway .
3. Another reason would be that the packet sent is larger than the path taken that the router can handle , the packet gets dropped.

The tcpdump command can be used to analyze the packets in a detailed manner .

3. An application reports: Connection timed out

DNS resolution works, but no response is received.

Identify which protocol layers could be responsible and justify each.

Ans A connection timed out means that the DNS resolution is working fine but no response is received this suggests that the TCP handshake is failing , the client sends the SYN signal but the server's response is not getting back successfully . The layers responsible for this :

1. Network Layer : The router or the device used to transmit the packet on the path is lost or due to the congestion on the device the packet is lost.
2. Application Layer : The client is sending the SYN signal but the server's SYN-ACK signal is not able to reach the client leading to connection timed out before the time limit. If the proxy is also used then a misconfigured proxy can cause the timeout issue.

4. Two networks have the following CIDRs:

Network A: 192.168.1.0/24

Network B: 192.168.1.128/25

Can these two networks communicate without NAT? Explain your answer.

Ans Yes , they will be able to communicate without NAT device , Network A is a larger network which includes all the IP addresses of Network B as well , so Basically Network B is a subset of Network A . Both of them can communicate without NAT because also they fall under the same private networks which makes the routing between them very straightforward.

5. A firewall rule allows inbound TCP traffic on port 443 but blocks all outbound traffic. Will HTTPS work? Explain in detail.

Ans No , HTTPS will not work , because for protocol like HTTPS to work it requires bidirectional communication to happen over the network . So since the firewall rule allows inbound traffic on port 443 but blocks all the outbound traffic , so the response sent by HTTPS is not reached and hence the connection fails . So to network connection to persist the stateful communication between client and server is required . Since the current rule blocks all outbound traffic the three way handshake fails.

6. After deploying a new version of an application:

DNS resolves correctly

TCP connection establishes

Requests hang indefinitely

What networking and application-level causes would you investigate before rolling back?

Ans : If the requests are hanging indefinitely after TCP connection establishes correctly then the issue lies within TLS negotiation or firewall rules are set in such way that the requests are not fulfilled correctly by the server.

The Network layer causes associated with it are that the packet which is sent is too large to send , so the TCP connection is opened , so the packets which are larger than the HTTP header are dropped silently . The TLS handshake may also be incomplete if the request

hangs immediately after TCP connection because the firewall may have a rule that blocks the outbound traffic from anywhere.

The Application-level causes can be :

- i) The new version can have a bug that exhausts the resource pool i.e requests resources in an abnormal way such that it overflow the TCP buffer to fulfill the requests and the packet data cannot be accommodated.
- ii) The new version of the application could lock any global resource which may cause race condition , causing all the incoming requests to queue indefinitely.
- iii) If the application require some response from different api or microservice whose URL is misconfigured and no timeout is set on that client request , then it hangs indefinitely.

7. On your Linux laptop, run `ip addr` and `ip route` . Using the actual output from your machine:

- a. Identify which network interface is currently used for internet connectivity and write its name.
- b. Write down your laptop's IP address, subnet mask, and default gateway for that interface.
- c. Based on the IP and subnet mask, calculate the network address, broadcast address, and number of usable host IPs in your local network. Show your working.

Ans a. `ip addr` command lists all the interfaces as a numbered block. So it shows three interfaces . The interface used for internet connectivity is `wlp0s20f3` which is my WIFI interface .

b. Laptop's IP address is : 192.168.1.84/24 , Subnet Mask is : 255.255.255.0 and Default Gateway is : 192.168.1.1 . The `ip route` command helps to do that.

c. The IP address of my system in binary form is `11000000.10101000.00000001.01010100` and subnet mask is `11111111.11111111.11111111.00000000` so using AND operation between them we can get the network address.

`11000000.10101000.00000001.01010100` IP

`11111111.11111111.11111111.00000000` Mask

`11000000.10101000.00000001.00000000`

In decimal format it is : 192.168.1.0

The broadcast address is the last address of the network which is reserved for broadcasting ,
192.168.1.255

Total Number of usable hosts are calculated as we have 24 bits as network bits so ,
Host bits = 32-24 = 8

$2^8 = 256$ hosts

The usable host IPs are 192.168.1.1 - 192.168.1.254

```
piyush@CK-piyush: ~  
piyush@CK-piyush:~$ ip route  
default via 192.168.1.1 dev wlp0s20f3 proto dhcp src 192.168.1.84 metric 600  
192.168.1.0/24 dev wlp0s20f3 proto kernel scope link src 192.168.1.84 metric 600  
piyush@CK-piyush:~$
```

```
piyush@CK-piyush:~$ ip addr  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host noprefixroute  
        valid_lft forever preferred_lft forever  
2: enp4s0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN group default qlen 1000  
    link/ether 88:a4:c2:95:ef:4f brd ff:ff:ff:ff:ff:ff  
3: wlp0s20f3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000  
    link/ether cc:15:31:e4:7c:4e brd ff:ff:ff:ff:ff:ff  
    inet 192.168.1.84/24 brd 192.168.1.255 scope global dynamic noprefixroute wlp0s20f3  
        valid_lft 85109sec preferred_lft 85109sec  
    inet6 fe80::4837:3ae8:760d:5bb5/64 scope link noprefixroute  
        valid_lft forever preferred_lft forever  
piyush@CK-piyush:~$
```

8. Choose a safe domain like [google.com] (http://google.com) .

- Show the current DNS resolution to google.com
- map google.com to 127.0.0.1 on your machine and explain how you did it.
- Show the DNS resolution after this change

Ans

a. To show current DNS resolution we can use `nslookup` command to see domain names resolved to IP addresses . For more detailed information `dig` command can be used . So I will use `nslookup google.com` to see to which IP address the domain is resolved too.

```
piyush@CK-piyush:~$ nslookup google.com  
Server:      127.0.0.53  
Address:     127.0.0.53#53  
  
Non-authoritative answer:  
Name:   google.com  
Address: 142.250.193.110  
Name:   google.com  
Address: 2404:6800:4002:808::200e
```

b. To map google.com to 127.0.0.1 on my machine I edited the `/etc/hosts` file on my machine and added 127.0.0.1 google.com as the entry and made the changes to the file.

```
piyush@CK-piyush: ~  
GNU nano 7.2 /etc/hosts  
127.0.0.1 localhost  
127.0.1.1 CK-piyush  
127.0.1.2 google.com  
# The following lines are desirable for IPv6 capable hosts  
::1 ip6-localhost ip6-loopback  
fe00::0 ip6-localnet  
ff00::0 ip6-mcastprefix  
ff02::1 ip6-allnodes  
ff02::2 ip6-allrouters
```

c. I mapped 127.0.0.2 to google.com and then I again used the `nslookup google.com` command to see that whether my machine resolve google.com to 127.0.0.2 before sending request to DNS server.

```
piyush@CK-piyush: ~$ nslookup google.com  
Server: 127.0.0.53  
Address: 127.0.0.53#53  
  
Name: google.com  
Address: 127.0.1.2  
  
piyush@CK-piyush: ~$
```

9. Start a simple HTTP server on your Linux laptop with `python3 -m http.server 8080` in any directory.

- In another terminal, take `tcpdump` of port 8080
- Make a request with `curl [http://localhost:8080](http://localhost:8080)` .
- Capture at least 8–10 lines of `tcpdump` output and:
 - Mark which packets are the TCP handshake (SYN, SYN/ACK, ACK).
 - Mark at least one packet carrying HTTP data.
 - Mark how the connection is closed (FIN/ACK etc., if visible)
- Explain how this sequence matches the theory of TCP connection setup, data transfer, and teardown.

Ans

```
piyush@CK-piyush: ~  
piyush@CK-piyush: ~  
piyush@CK-piyush: ~  
piyush@CK-piyush: ~$ python3 -m http.server 8080  
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
```

- The command I first used was `tcpdump 8080` but this denied the output because it requires `sudo` privileges to see the traffic. So I used the following command

`sudo tcpdump -i wlp0s20f3 port 8080 -vv` where I used the specific network interface which was previously not giving the output and used `-i` flag for the interface to watch and `-vv` to give verbose output.

```
(No such device exists)
piyush@CK-piyush:~$ sudo tcpdump -i wlp0s20f3 port 8080 -vv
tcpdump: listening on wlp0s20f3, link-type EN10MB (Ethernet), snapshot length 262144 bytes
```

b.

```
piyush@CK-piyush:~$ curl localhost:8080
<!DOCTYPE HTML>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Directory listing for /</title>
</head>
<body>
<h1>Directory listing for /</h1>
<hr>
<ul>
<li><a href=".bash_history">.bash_history</a></li>
<li><a href=".bash_logout">.bash_logout</a></li>
<li><a href=".bashrc">.bashrc</a></li>
<li><a href=".cache/">.cache/</a></li>
<li><a href=".config/">.config/</a></li>
<li><a href=".gnupg/">.gnupg/</a></li>
<li><a href=".lessht">.lessht</a></li>
<li><a href=".local/">.local/</a></li>
<li><a href=".pki/">.pki/</a></li>
<li><a href=".profile">.profile</a></li>
<li><a href=".selected_editor">.selected_editor</a></li>
<li><a href=".ssh/">.ssh/</a></li>
<li><a href=".sudo_as_admin_successful">.sudo_as_admin_successful</a></li>
<li><a href=".wget-hsts">.wget-hsts</a></li>
<li><a href="Desktop/">Desktop/</a></li>
<li><a href="Documents/">Documents/</a></li>
<li><a href="Downloads/">Downloads/</a></li>
<li><a href="exercise/">exercise/</a></li>
<li><a href="input.txt">input.txt</a></li>
```

```
<li><a href="input.txt">input.txt</a></li>
<li><a href="Music/">Music/</a></li>
<li><a href="output.txt">output.txt</a></li>
<li><a href="Pictures/">Pictures/</a></li>
<li><a href="Public/">Public/</a></li>
<li><a href="snap/">snap/</a></li>
<li><a href="Templates/">Templates/</a></li>
<li><a href="Videos/">Videos/</a></li>
</ul>
<hr>
</body>
</html>
```

C.

```
piyush@CK-piyush:~$ sudo tcpdump -i lo to tcp port 8080 -nv
tcpdump: listening on lo, link-type EN10MB (Ethernet), snapshot length 262144 bytes
10:29:44.658913 IP6 (Flowlabel 0xf78eb, hlim 64, next-header TCP (6) payload length: 40) ::1.42986 > ::1.8080: Flags [S], cksum 0x0030 (incorrect -> 0xf767), seq 1607702648, win 65476, options [mss 65476,sackOK,TS val 1691628070 ecr 0,nop,wscale 7], length 0
10:29:44.658924 IP6 (Flowlabel 0xbca69, hlim 64, next-header TCP (6) payload length: 20) ::1.8080 > ::1.42986: Flags [R.], cksum 0x001c (incorrect -> 0xf007), seq 0, ack 1607702649, win 0, length 0
10:29:44.659090 IP (tos 0x0, ttl 64, id 58937, offset 0, flags [DF], proto TCP (6), length 60)
  127.0.0.1.40932 > 127.0.0.1.8080: Flags [S], cksum 0xfe30 (incorrect -> 0x4b96), seq 625353042, win 65495, options [mss 65495,sackOK,TS val 1678809678 ecr 0,nop,wscale 7], length 0
10:29:44.659015 IP (tos 0x0, ttl 64, id 0, offset 0, flags [DF], proto TCP (6), length 60)
  127.0.0.1.8080 > 127.0.0.1.40932: Flags [S.], cksum 0xfe30 (incorrect -> 0xb57a), seq 3993872809, ack 625353043, win 65483, options [mss 65495,sackOK,TS val 1678809678 ecr 1678809678,nop,wscale 7], length 0
10:29:44.659030 IP (tos 0x0, ttl 64, id 58938, offset 0, flags [DF], proto TCP (6), length 52)
  127.0.0.1.40932 > 127.0.0.1.8080: Flags [.], cksum 0xfe28 (incorrect -> 0xdc36), seq 1, ack 1, win 512, options [nop,nop,TS val 1678809678 ecr 1678809678], length 0
10:29:44.659090 IP (tos 0x0, ttl 64, id 58939, offset 0, flags [DF], proto TCP (6), length 129)
  127.0.0.1.40932 > 127.0.0.1.8080: Flags [P.], cksum 0xfe75 (incorrect -> 0x58a4), seq 1:78, ack 1, win 512, options [nop,nop,TS val 1678809678 ecr 1678809678], length 77: HTTP, length: 77
  GET / HTTP/1.1
  Host: localhost:8080
  User-Agent: curl/8.5.0
  Accept: /*
10:29:44.659096 IP (tos 0x0, ttl 64, id 48412, offset 0, flags [DF], proto TCP (6), length 52)
  127.0.0.1.8080 > 127.0.0.1.40932: Flags [.], cksum 0xfe28 (incorrect -> 0xdbea), seq 1, ack 78, win 511, options [nop,nop,TS val 1678809678 ecr 1678809678], length 0
10:29:44.660646 IP (tos 0x0, ttl 64, id 48413, offset 0, flags [DF], proto TCP (6), length 208)
  127.0.0.1.8080 > 127.0.0.1.40932: Flags [P.], cksum 0xfec4 (incorrect -> 0xfa96), seq 1:157, ack 78, win 512, options [nop,nop,TS val 1678809679 ecr 1678809678], length 156: HTTP, length: 156
  HTTP/1.0 200 OK
  Server: SimpleHTTP/0.6 Python/3.12.3
  Date: Sat, 24 Jan 2026 04:59:44 GMT
  Content-type: text/html; charset=utf-8
  Content-Length: 1313
10:29:44.660664 IP (tos 0x0, ttl 64, id 58940, offset 0, flags [DF], proto TCP (6), length 52)
  127.0.0.1.40932 > 127.0.0.1.8080: Flags [.], cksum 0xfe28 (incorrect -> 0xdb4c), seq 78, ack 157, win 511, options [nop,nop,TS val 1678809679 ecr 1678809679], length 0
10:29:44.660698 IP (tos 0x0, ttl 64, id 48414, offset 0, flags [DF], proto TCP (6), length 1365)
  127.0.0.1.8080 > 127.0.0.1.40932: Flags [P.], cksum 0x834a (incorrect -> 0x78fa), seq 157:1470, ack 78, win 512, options [nop,nop,TS val 1678809679 ecr 1678809679], length 1313: HTTP
  p
10:29:44.660706 IP (tos 0x0, ttl 64, id 58941, offset 0, flags [DF], proto TCP (6), length 52)
  127.0.0.1.40932 > 127.0.0.1.8080: Flags [.], cksum 0xfe28 (incorrect -> 0xd5e8), seq 78, ack 1470, win 578, options [nop,nop,TS val 1678809679 ecr 1678809679], length 0
10:29:44.660751 IP (tos 0x0, ttl 64, id 48415, offset 0, flags [DF], proto TCP (6), length 52)
  127.0.0.1.8080 > 127.0.0.1.40932: Flags [F.], cksum 0xfe28 (incorrect -> 0xd629), seq 1470, ack 78, win 512, options [nop,nop,TS val 1678809679 ecr 1678809679], length 0
10:29:44.660848 IP (tos 0x0, ttl 64, id 58942, offset 0, flags [DF], proto TCP (6), length 52)
  127.0.0.1.40932 > 127.0.0.1.8080: Flags [.], seq 1470, ack 1470, win 0, length 0
```

i.) The SYN Packet is marked as , it is marked by [S] :

127.0.0.1.40932 > 127.0.0.1.8080: Flags [S], seq 625353042

The SYN-ACK Packet sent by server is it is marked by [S.]:

127.0.0.1.8080 > 127.0.0.1.40932: Flags [S.], seq 3993872809, ack 625353043

The ACK Packet sent again by client is and marked by [.] :

127.0.0.1.40932 > 127.0.0.1.8080: Flags [.] , ack 1

The above three way handshake shows that connection is established successfully.

ii.) The packet carrying HTTP data is shown by the flag [P.] :

127.0.0.1.8080 > 127.0.0.1.40932: Flags [P.], seq 157:1470, length 1313: HTTP

iii) The connection closer is shown by the flag [F.] finish flag :

127.0.0.1.8080 > 127.0.0.1.40932: Flags [F.], seq 1470

The client acknowledges the FIN signal of server [.] :

127.0.0.1.40932 > 127.0.0.1.8080: Flags [.] , ack 1470

d. This sequence matches the theory of TCP connection as :

First the Client sent the SYN signal which is clearly visible , the server responds by sending SYN-ACK signal and then the client sends ACK signal that the client acknowledges the signal received which shows the connection is established.

Now the Data transfer is shown , after the successful connection where Data is carried inside TCP segments marked with [P] flag , HTTP get request is sent from client using curl and server response with HTTP 200 OK + HTML response from the server.

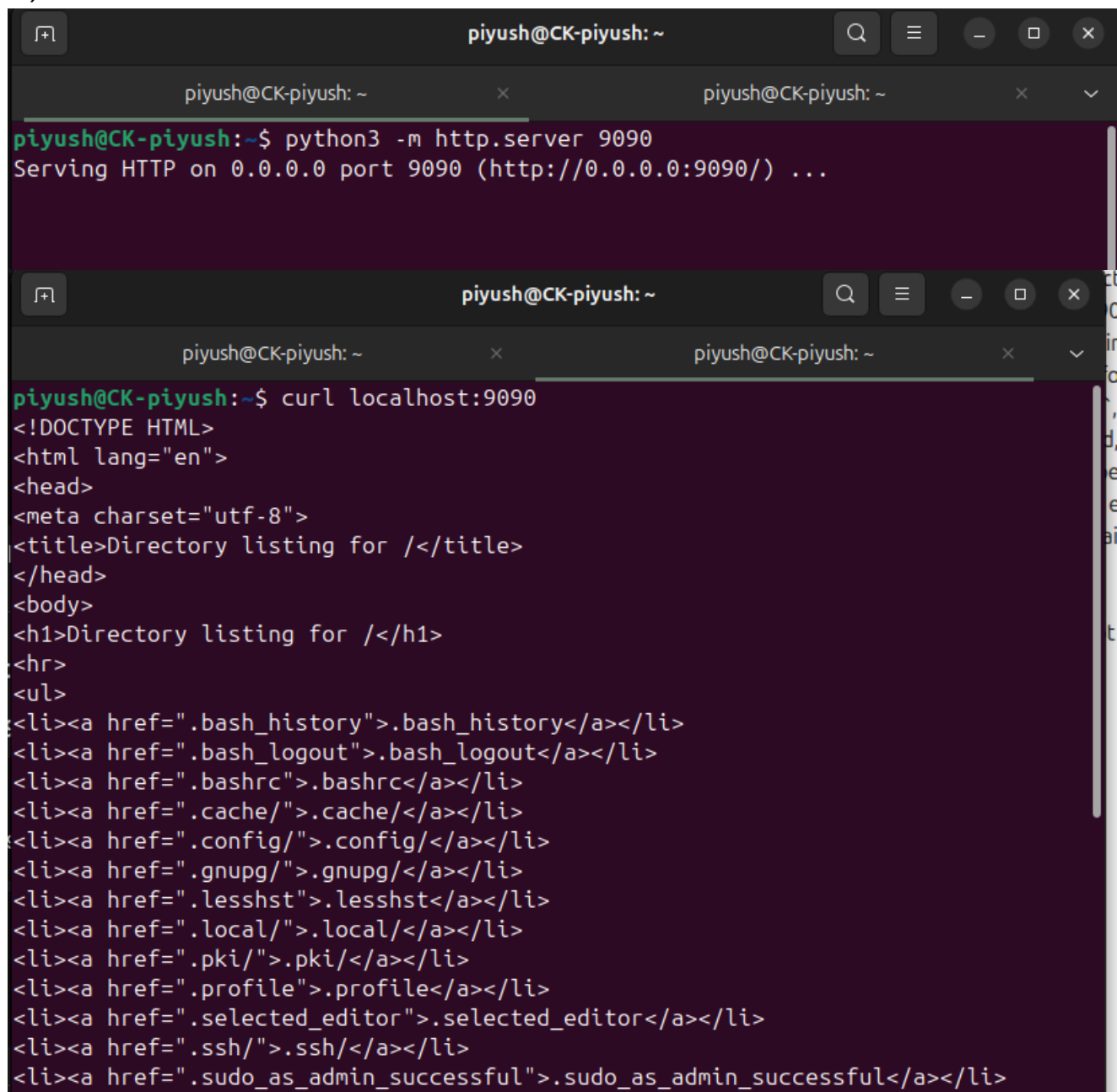
TCP connection is closed using FIN/ACK signal. The server after serving the request sends FIN i.e finish signal and the client sent the ACK signal to close the connection.

10. Intentionally break connectivity to a local service and debug it.

- Start any service listening on a TCP port (for example, `python3 -m http.server 9090`) and verify `curl localhost:9090` works.
- Break connectivity by adding a local firewall rule using `ufw` or `iptables` that blocks port 9090.
- Show the failing request (command + output).
- Use at least four commands from the slides (for example: `ss -tulnp` or `netstat -tulnp`, `ip addr`, `ip route`, `curl -v`, `ping`, `traceroute`) to diagnose the issue. Paste the outputs and, for each command, write 2–3 sentences explaining what it told you and how it helped narrow down the root cause.
- Describe the exact change you made to fix the problem and show that the service is reachable again (command + output).

Ans

a.)



```
piyush@CK-piyush: ~  
piyush@CK-piyush: ~$ python3 -m http.server 9090  
Serving HTTP on 0.0.0.0 port 9090 (http://0.0.0.0:9090/) ...  
  
piyush@CK-piyush: ~$ curl localhost:9090  
<!DOCTYPE HTML>  
<html lang="en">  
<head>  
<meta charset="utf-8">  
<title>Directory listing for /</title>  
</head>  
<body>  
<h1>Directory listing for /</h1>  
<hr>  
<ul>  
<li><a href=".bash_history">.bash_history</a></li>  
<li><a href=".bash_logout">.bash_logout</a></li>  
<li><a href=".bashrc">.bashrc</a></li>  
<li><a href=".cache/">.cache/</a></li>  
<li><a href=".config/">.config/</a></li>  
<li><a href=".gnupg/">.gnupg/</a></li>  
<li><a href=".lesshtst">.lesshtst</a></li>  
<li><a href=".local/">.local/</a></li>  
<li><a href=".pki/">.pki/</a></li>  
<li><a href=".profile">.profile</a></li>  
<li><a href=".selected_editor">.selected_editor</a></li>  
<li><a href=".ssh/">.ssh/</a></li>  
<li><a href=".sudo_as_admin_successful">.sudo_as_admin_successful</a></li>
```

So I ran `python -m http.server 9090` on one terminal to run a local server and then on another terminal I use `curl localhost 9090` to verify the service is up.

b)

```
piyush@CK-piyush:~$ sudo ufw deny 9090/tcp
[sudo] password for piyush:
Rules updated
Rules updated (v6)
```

Using the command `sudo ufw deny 9090/tcp` I added the firewall rule to deny the connection request on port 9090.

c) By running the command `curl -v localhost:9090` I see the now my firewall blocks the request from the client.

```
piyush@CK-piyush:~$ curl localhost:9090
^C
piyush@CK-piyush:~$ curl -v localhost:9090
* Host localhost:9090 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
* Trying [::1]:9090...
* connect to ::1 port 9090 from ::1 port 33474 failed: Connection refused
```

d) Using `ss -tulpn`

It shows all active TCP connections and their status .

```
piyush@CK-piyush:~$ ss -tulpn | grep 9090
tcp    LISTEN 0      5            0.0.0.0:9090      0.0.0.0:*        users:(("python
3",pid=3703,fd=3))
```

Using `iptables -L`

It will show all the firewall rules and which port have allowed the traffic and which deny it .

```
piyush@CK-piyush:~$ sudo iptables -L
Chain INPUT (policy DROP)
target     prot opt source                destination           tcp dpt:9090
DROP       tcp  --  anywhere              anywhere              tcp dpt:9090
DROP       tcp  --  anywhere              anywhere              tcp dpt:9090
ufw-before-logging-input all  --  anywhere              anywhere
ufw-before-input all  --  anywhere              anywhere
ufw-after-input all  --  anywhere              anywhere
ufw-after-logging-input all  --  anywhere              anywhere
ufw-reject-input all  --  anywhere              anywhere
ufw-track-input all  --  anywhere              anywhere
REJECT     tcp  --  anywhere              anywhere              tcp dpt:9090 reject
t-with icmp-port-unreachable

Chain FORWARD (policy DROP)
target     prot opt source                destination
ufw-before-logging-forward all  --  anywhere              anywhere
ufw-before-forward all  --  anywhere              anywhere
ufw-after-forward all  --  anywhere              anywhere
ufw-after-logging-forward all  --  anywhere              anywhere
ufw-reject-forward all  --  anywhere              anywhere
ufw-track-forward all  --  anywhere              anywhere
```

Using `netstat -tunlp`

This command list all the ports which are currently listening and which are not allowed .

```

piyush@CK-piyush:~$ netstat -tunlp
(Not all processes could be identified, non-owned process info
will not be shown, you would have to be root to see it all.)
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
PID/Program name
tcp        0      0 0.0.0.0:9090            0.0.0.0:*               LISTEN
3703/python3
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:80              0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.1:631           0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.54:53           0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.53:53           0.0.0.0:*               LISTEN
tcp6       0      0 :::22                  :::*                    LISTEN
tcp6       0      0 :::80                  :::*                    LISTEN

```

Using `traceroute localhost`

This command shows the network path to localhost .

```

piyush@CK-piyush:~$ traceroute localhost
traceroute to localhost (127.0.0.1), 64 hops max
 1  127.0.0.1  0.005ms  0.002ms  0.001ms

```

e) I delete the rules which were causing the trouble and not allowing the port 9090 to response .

Using command `sudo ufw delete deny 9090/tcp`

```

piyush@CK-piyush:~$ sudo ufw status numbered
Status: active

    To      Action     From
    --      -
[ 1] 9090/tcp DENY IN     Anywhere
[ 2] 9090/tcp on lo  ALLOW IN   Anywhere
[ 3] 9090/tcp (v6)  DENY IN    Anywhere (v6)
[ 4] 9090/tcp (v6) on lo  ALLOW IN   Anywhere (v6)
[ 5] 9090/tcp on lo  ALLOW IN   ::1

```

```

piyush@CK-piyush:~$ sudo ufw status
Status: active

To      Action     From
--      -
9090/tcp on lo  ALLOW      Anywhere
9090/tcp (v6) on lo  ALLOW      Anywhere (v6)

```

Now the `curl localhost:9090` gives the response received from the server.

11. Run `traceroute google.com` (or `mtr google.com` if available) from your Linux laptop.

1. Paste the full output.
2. Identify one hop that is likely your home/office router or first upstream device, and one hop where latency increases significantly compared to the previous hop.
3. For each of these two hops, explain what you think it represents in the real world (for example, local router, ISP edge, backbone router), using IP ranges, hostnames (if visible), and RTT patterns to justify your answer.
4. Comment on whether the overall latency is acceptable for a typical web application and why.

Ans

1.)

```
piyush@CK-piyush:~$ traceroute google.com
traceroute to google.com (142.250.193.110), 64 hops max
 1  192.168.1.1  1.267ms  1.154ms  1.674ms
 2  * * *
 3  * * *
 4  * * *
 5  * * *
 6  * * *
 7  * * *
 8  * * *
 9  142.250.193.110  6.191ms  2.538ms  2.611ms
```

2.) 192.168.1.1 is my wifi router . The latency increases at Hop 9 which is the destination of server google.com .

3.) 192.168.1.1 is my wifi router . It is device from which the packets first go through the local network. Its IP address is in the private network range for internal networks.

142.250.193.110 is the final destination sever of Google. My hops from 2-8 are '***' because the firewall may be blocking some responses or the network may be congested .

4.) The overall latency for web applications under 100ms is acceptable. In my case the average latency across the network is 1.5 to 3 ms which is excellent for web browsing or typical web applications . The latency should not affect real time applications such as video applications such as video streaming or VoIP but its acceptable for a web application .